

P2299R2

mdspan and CTAD

Published Proposal, 2021-05-21

Author:

[Bryce Adelstein Lelbach \(he/him/his\) — Library Evolution Chair](#) (NVIDIA)

Source:

[GitHub](#)

Issue Tracking:

[GitHub](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

WG21

Table of Contents

1 Introduction

2 Wording

References

Informative References

§ 1. Introduction

[P0009R11] proposes adding non-owning multidimensional span abstractions to the C++ Standard Library; `basic_mdspan`, which is fully generic and can represent any kind of multidimensional data, and `mdspan`, a convenience template alias for simpler use cases.

```
template <class ElementType, class Extents, class LayoutPolicy = layout_right,
         class AccessorPolicy = accessor_basic<ElementType>>
struct basic_mdspan;
```

```
template <class ElementType, size_t... Extents>
using mdspan = basic_mdspan<ElementType, extents<Extents...>>;
```

In the `basic_mdspan/span` interface, extents can be either static, e.g. expressed at compile time:

```
mdspan<double, 64, 64> a(data);
```

or dynamic, e.g. expressed at run time:

```
mdspan<double, dynamic_extent, dynamic_extent> a(data, 64, 64);
```

You can also use a mix of the two styles:

```
mdspan<double, 64, dynamic_extent> a(data, 64);
```

While `basic_mdspan` and `mdspan` are powerful, the spelling of instantiations can be verbose, especially for the common case where all extents are dynamic.

Using class template argument deduction (introduced in C++17) and alias template argument deduction (introduced in C++20), we can make it easier to use `mdspan` with all dynamic extents:

Before	After
<code>mdspan<double, dynamic_extent, dynamic_extent> a(data, 64, 64);</code>	<code>mdspan a(data, 64, 64);</code>

To make this work, we need to add a deduction guide for `basic_mdspan`. Through the power of alias template argument deduction, `mdspan` will be able to use said deduction guide as well.

Here's [an example](#) implementation of such a deduction guide for `basic_mdspan`.

In earlier versions of this paper, it was unclear whether such a deduction guide would work for the alias template `mdspan`, as attempts to construct one that functioned as intended had failed. However, we have since learned that this is due to bugs in the two implementations that currently support

alias template deduction, MSVC and GCC. Those bugs have been reported to the respective compilers and hopefully will be fixed soon.

Here's [an example](#) of the current implementation bugs that prevent the deduction guide from working for `mdspan`.

§ 2. Wording

The following changes are relative to the `mdspan` proposal ([\[P0009R11\]](#)).

The  character is used to denote a placeholder number which shall be selected by the editor.

Modify the header synopsis for `<mdspan>` in [**`mdspan.syn`**] as follows:

```

namespace std {
    // [mdspan.extents], class template extents
    template
        class extents;

    // [mdspan.layout], Layout mapping policies
    class layout_left;
    class layout_right;
    class layout_stride;

    // [mdspan.accessor.basic]
    template<class ElementType>
        class default_accessor;

    // [mdspan.basic], class template mdspan
    template<class ElementType, class Extents, class LayoutPolicy = layout_
                class AccessorPolicy = default_accessor<ElementType>>
        class basic_mdspan;

    template <class ElementType, class... IndexTypes>
    explicit basic_mdspan(ElementType*, IndexTypes...)
        -> basic_mdspan<ElementType, extents<[] (auto) constexpr
                                { return dynamic_extent; }
                                (identity<IndexTypes>{})...>>;

    template<class T, size_t... Extents>
        using mdspan = basic_mdspan<T, extents<Extents...>>;

    // [mdspan.submdspan]
    template<class ElementType, class Extents, class LayoutPolicy,
            class AccessorPolicy, class... SliceSpecifiers>
        constexpr basic_mdspan<see below>
        submdspan(const basic_mdspan<ElementType, Extents, LayoutPolicy, Acce
            SliceSpecifiers... specs) noexcept;

    // tag supporting submdspan
    struct full_extent_t { explicit full_extent_t() = default; };
    inline constexpr full_extent_t full_extent = full_extent_t{};
}

```

§ References

§ Informative References

[P0009R11]

Christian Trott; et al. `mdspan`. 2020-05-11. URL:
<https://isocpp.org/files/papers/P0009R11.html>